```
'''carplanner_solution.py

   This is the solution code for the car planner.

   Use and EST/Dijkstra combination to plan car movements.
'''

import bisect
import matplotlib.pyplot as plt
import numpy as np
import random
import time

from math               import pi, sin, cos, tan, atan2, sqrt, ceil, floor
from shapely.geometry   import Point, MultiPoint
from shapely.geometry   import LineString, MultiLineString
from shapely.geometry   import Polygon, MultiPolygon
from shapely.prepared   import prep


##########################################################################
# SECTION 1: PARAMETER DEFINITIONS
##########################################################################
#
#   Algorithm Parameters
#
#   Scenario.  Choose from:
#     'parallel parking'        HW7 Problem 4
#     'u turn'                  HW7 Problem 5
#     'move over'               HW7 Problem 6
#
#   Costs:
#     cstep             Cost per each step
#     csteer            Cost for each change in steering angle
#     creverse          Cost for each change in fworward/back direction
#
#   Grid Sizes:
#     dstep             Distance (meters) per grid cell
#     thetastep         Angle (radian) per grid cell
#
#   FIXME: Please change the scenario and the costs as you explore
#

# Select the scenario.
```

```
45  # scenario = 'parallel parking'
46  #scenario = 'u turn'
47  scenario = 'move over'
48
49  # Choose the cost coefficients.
50  cstep    =   1
51  csteer   =  10
52  creverse = 10
53
54  # Choose the grid sizes.
55  if scenario == 'parallel parking':
56      dstep     = 0.4             # Distrance driven per move
57      thetastep = pi/36           # Angle rotated per non-straight move
58
59  elif scenario == 'u turn':
60      dstep     = 0.5             # Distrance driven per move
61      thetastep = pi/36           # Angle rotated per non-straight move
62
63  elif scenario == 'move over':
64      dstep     = 0.5             # Distrance driven per move
65      thetastep = pi/36           # Angle rotated per non-straight move
66
67  else:
68      raise ValueError("Unknown Scenario")
69
70
71  ##########################################################################
72  #
73  #   Car Definitions
74  #
75  #   Define the car dimensions and parameters.
76  #
77
78  # Car outline.
79  (lcar, wcar) = (4, 2)           # Length/width
80
81  # Center of rotation (center of rear axle).
82  lb        = 0.5                 # Center of rotation to back bumper
83  lf        = lcar - lb           # Center of rotation to front bumper
84  wc        = wcar/2              # Center of rotation to left/right
85  wheelbase = 3                   # Center of rotation to front wheels
86
87
88  ##########################################################################
89  #
90  #   World Defintions
```

```
#
#   This should select:
#
#   (xmin, xmax, ymin, ymax)        # Overall dimensions
#   (xstart, ystart, thetastart)    # Starting pose
#   (xgoal,  ygoal,  thetagoal)     # Goal pose
#   walls                           # Shapely MultiLineString object
#

### Parallel Parking:
if scenario == 'parallel parking':

    # General numbers.
    (wroad)                  = 6                # Road width
    (xspace, lspace, wspace) = (3, 6, 2.5)      # Parking Space pos/len/width

    # Overall size.
    (xmin, xmax) = (0, 14)
    (ymin, ymax) = (0, wroad+wspace)

    # Pick your start and goal locations.
    (xstart, ystart, thetastart) = (1.0, 4.0, 0.0)
    (xgoal,  ygoal,  thetagoal)  = (xspace+(lspace-lcar)/2+lb, wroad+wc, 0.0)

    # Define the walls.
    walls = prep(MultiLineString([LineString([
        [xmin, ymin], [xmax, ymin], [xmax, wroad], [xspace+lspace, wroad],
        [xspace+lspace, wroad+wspace], [xspace, wroad+wspace],
        [xspace, wroad], [xmin, wroad], [xmin, ymin]])]))

### U Turn (3 point turn).
elif scenario == 'u turn':

    # General numbers.
    (xroad, yroad, wlane) = (5, 10, 3)  # road len, pos, lane width

    # Overall size.
    (xmin, xmax) = (0, 22)
    (ymin, ymax) = (0, 20)

    # Pick your start and goal locations.
    (xstart, ystart, thetastart) = (1.0, yroad-1.5, 0)
    (xgoal,  ygoal,  thetagoal)  = (4.0, yroad+1.5, pi)

    # Define the walls.
    walls = prep(MultiLineString([LineString([
```

```
137         [xmin, yroad-wlane], [xroad, yroad-wlane], [xroad, ymin],
138         [xmax, ymin], [xmax, ymax], [xroad, ymax], [xroad, yroad+wlane],
139         [xmin, yroad+wlane], [xmin, yroad-wlane]])]))
140
141 ### Move/Shift over
142 elif scenario == 'move over':
143
144     # General numbers.
145     (xrail, yrail, lrail) = (5, 4, 8)          # Guard rail pos/len
146
147     # Overall size.
148     (xmin, xmax) = (0, 20)
149     (ymin, ymax) = (0, 20)
150
151     # Pick your start and goal locations.
152     (xstart, ystart, thetastart) = (9, 2.0, 0.0)
153     (xgoal,  ygoal,  thetagoal)  = (9, 6.0, 0.0)
154
155     # Define the walls.
156     walls = prep(MultiLineString([
157         LineString([[xmin, ymin], [xmax, ymin], [xmax, ymax],
158                     [xmin, ymax], [xmin, ymin]]),
159         LineString([[xrail, yrail], [xrail+lrail, yrail]])]))
160
161 ### Unknown.
162 else:
163     raise ValueError("Unknown Scenario")
164
165
166 ##########################################################################
167 # SECTION 2: VISUALIZATION
168 ##########################################################################
169 #
170 #   Visualization Utility Class
171 #
172 class Visualization:
173     def __init__(self):
174         # Clear the current, or create a new figure.
175         plt.clf()
176
177         # Create a new axes, enable the grid, and set axis limits.
178         plt.axes()
179         plt.grid(True)
180         plt.gca().axis('on')
181         plt.gca().set_xlim(xmin, xmax)
182         plt.gca().set_ylim(ymin, ymax)
```

```
183         plt.gca().set_aspect('equal')
184 
185         # Show the walls.
186         xticks = set([xstart, xgoal])
187         yticks = set([ystart, ygoal])
188         for lines in walls.context.geoms:
189             xticks.update(lines.xy[0])
190             yticks.update(lines.xy[1])
191             plt.plot(lines.xy[0], lines.xy[1], 'k-', linewidth=2)
192         plt.gca().set_xticks(list(xticks))
193         plt.gca().set_yticks(list(yticks))
194 
195         # Show.
196         self.show()
197 
198     def show(self, text = '', delay = 0.001):
199         # Show the plot.
200         plt.pause(max(0.001, delay))
201         # If text is specified, print and maybe wait for confirmation.
202         if len(text)>0:
203             if delay>0:
204                 input(text + ' (hit return to continue)')
205             else:
206                 print(text)
207 
208     def drawNode(self, node, **kwargs):
209         # Show the box/frame.
210         plt.plot(*node.box.exterior.xy, **kwargs)
211         # Add headlights set in 10% from front corners.
212         (flx,fly) = node.box.exterior.coords[0]
213         (frx,fry) = node.box.exterior.coords[3]
214         plt.plot(0.9*frx+0.1*flx, 0.9*fry+0.1*fly, **kwargs, marker='o')
215         plt.plot(0.1*frx+0.9*flx, 0.1*fry+0.9*fly, **kwargs, marker='o')
216 
217     def drawEdge(self, head, tail, **kwargs):
218         plt.plot([head.x, tail.x], [head.y, tail.y], **kwargs)
219 
220     def drawPath(self, path, **kwargs):
221         for node in path:
222             self.drawNode(node, **kwargs)
223             self.show(delay = 0.1)
224 
225 
226 
227 ##############################################################################
228 # SECTION 3: NODE
```

```
229 ########################################################################
230 #
231 #   Node Definition
232 #
233 # Initialize the counters (for diagnostics only)
234 nodecounter = 0
235 donecounter = 0
236
237 # Make sure thetastep is an integer fraction of 2pi, so the grid wraps nicely.
238 #thetastep = dstep * tan(steerangle) / wheelbase
239 thetastep  = 2*pi / round(2*pi/thetastep)
240 steerangle = np.arctan(wheelbase * thetastep / dstep)
241
242 # Node Class
243 class Node:
244     def __init__(self, x, y, theta):
245         # Setup the basic A* node.
246         super().__init__()
247
248         # Remember the state (x,y,theta).
249         self.x     = x
250         self.y     = y
251         self.theta = theta
252
253         # Box = 4 corners: frontleft, backleft, backright, frontright
254         (s,c) = (sin(theta), cos(theta))
255         self.box = Polygon([[x + c*lf - s*wc, y + s*lf + c*wc],
256                             [x - c*lb - s*wc, y - s*lb + c*wc],
257                             [x - c*lb + s*wc, y - s*lb - c*wc],
258                             [x + c*lf + s*wc, y + s*lf - c*wc]])
259
260         # Tree connectivity.  Define how we got here (set defaults for now).
261         self.parent  = None
262         self.forward = 1        # +1 = drove forward, -1 = backward
263         self.steer   = 0        # +1 = turned left, 0 = straight, -1 = right
264
265         # Cost/status.
266         self.cost = 0           # Cost to get here.
267         self.done = False       # The path here is optimal.
268
269         # Count the node - for diagnostics only.
270         global nodecounter
271         nodecounter += 1
272         if nodecounter % 1000 == 0:
273             print("Sampled %d nodes... fully processed %d nodes" %
274                   (nodecounter, donecounter))
```

```
275
276         ############
277         # Utilities:
278         # In case we want to print the node.
279         def __repr__(self):
280             return ("<XY %7.4f,%7.4f @ %5.1f deg> (fwd %2d, str %2d, cost %4d)" %
281                     (self.x, self.y, np.degrees(self.theta),
282                      self.forward, self.steer, self.cost))
283
284         # Define the "less-than" to enable sorting by cost!
285         def __lt__(self, other):
286             return (self.cost < other.cost)
287
288         # Determine the grid indices based on the coordinates.  The x/y
289         # coordinates are regular numbers, the theta maps to 0..360deg!
290         def indices(self):
291             return (round((self.x - xmin)/dstep),
292                     round((self.y - ymin)/dstep),
293                     round(self.theta/thetastep) % round(2*pi/thetastep))
294
295
296         ######################
297         # Forward Simulations
298         # Instantiate a new (child) node, based on driving from this node.
299         def nextNode(self, forward, steer):
300             # Assume forward = +1,    -1    (sign of d)
301             #        steer   = +1, 0, -1    (sign of steering, NOT of deltatheta)
302
303             # Check if car is going straight or turning and calculate xb, yb, and thetab
304             if steer == 0:
305                 d = forward * dstep     # Calculate whether car is going forward or backward
306                 xnext = self.x + d * cos(self.theta)
307                 ynext = self.y + d * sin(self.theta)
308                 thetanext = self.theta
309             else:
310                 d = forward * dstep
311                 thetadelta = forward * steer * thetastep    # # Calculate whether thetadelta is
312                 R = d / thetadelta
313                 thetanext = (self.theta + thetadelta + pi) % (2 * pi) - pi  # Symmetrical wrap
314                 xnext = self.x - R * sin(self.theta) + R * sin(thetanext)
315                 ynext = self.y + R * cos(self.theta) - R * cos(thetanext)
316
317             # Create the child node.
318             child = Node(xnext, ynext, thetanext)
319
320             # Set the parent relationship.
```

```
321             child.parent  = self
322             child.forward = forward
323             child.steer   = steer
324 
325             # Calculate the updated cost for child node
326             Nstep = 1
327             # Penalize if steer changed
328             if steer != self.steer:
329                 Nsteer = 1
330             else:
331                 Nsteer = 0
332 
333             # Penalize for changing direction
334             if forward != self.forward:
335                 Nrev = 1
336             else:
337                 Nrev = 0
338 
339             cost_add = cstep * Nstep + csteer * Nsteer + creverse * Nrev
340             child.cost = self.cost + cost_add
341             return child
342 
343 
344     ######################
345     # Collision functions:
346     # Check whether in free space.
347     def inFreespace(self):
348         return walls.disjoint(self.box)
349 
350     # Check the local planner - whether this connects to another node.
351     # This is slightly approximate, but good for the small steps.
352     def connectsTo(self, other):
353         return walls.disjoint(self.box.union(other.box).convex_hull)
354 
355 
356 ##########################################################################
357 # SECTION 4: PLANNER
358 ##########################################################################
359 #
360 #   Planner Functions
361 #
362 def planner(startnode, goalnode):
363     # Create the grid to store one Node per square.  Add 1 to the x/y
364     # dimensions to include min and max values.  See Node.indices().
365     grid = np.empty((1 + int((xmax - xmin) / dstep),
366                      1 + int((ymax - ymin) / dstep),
```

```
367                     round(2*pi / thetastep)), dtype='object')
368         print("Created %dx%dx%d grid (with %d elements)" %
369               (np.shape(grid) + (np.size(grid),)))
370 
371         # Prepare the still empty *sorted* on-deck queue.
372         onDeck = []
373 
374         # Begin with the start node on-deck and in the grid.
375         grid[startnode.indices()] = startnode
376         bisect.insort(onDeck, startnode)
377 
378         # Continually expand/build the search tree.
379         while True:
380             # Make sure we have something pending in the on-deck queue.
381             # Otherwise we were unable to find a path!
382             if not (len(onDeck) > 0):
383                 return None
384 
385             # Grab the next node (first on deck).
386             node = onDeck.pop(0)
387 
388             # Mark this node as done.
389             node.done = True
390             global donecounter
391             donecounter += 1
392 
393             if node.indices() == goalnode.indices():
394                 break
395 
396             # Loop through all possible actions
397             for forward in (-1, +1):
398                 for steer in (+1, 0, -1):
399                     child = node.nextNode(forward, steer)
400                     ind = child.indices()
401                     if ind == node.indices():
402                         child = child.nextNode(forward, steer)
403                         ind = child.indices()
404 
405                     if child.inFreespace() and node.connectsTo(child):
406                         if grid[ind] is None:
407                             grid[ind] = child
408                             bisect.insort(onDeck, child)
409                         else:
410                             prev_node = grid[ind]
411                             if (prev_node.done == False) and (child.cost < prev_node.cost):
412                                 onDeck.remove(prev_node)
```

```
                                grid[ind] = child
                                bisect.insort(onDeck, child)

            # Break the loop if done.

        # Build the path.
        path = [node]
        while path[0].parent is not None:
            path.insert(0, path[0].parent)

        # Return the path.
        return path


#########################################################################
# SECTION 5: TESTING AND MAIN FUNCTION
#########################################################################
#
#  Test Functions
#
def testdriving(visual):
    # Test with three repeated action.

    def testdrive1(visual, node, forward, steer, color, case, N=3):
        visual.drawNode(node, color='k', linewidth=2)
        print(node, " before ", case)
        for i in range(N):
            node = node.nextNode(forward, steer)
            visual.drawNode(node, color=color, linewidth=2)
        print(node, " after", N, case)

    # Try all six actions.
    testdrive1(visual, Node(8.0, 1.0, 0.0),  1.0,  1.0, 'b', 'L+')
    testdrive1(visual, Node(8.0, 4.0, 0.0),  1.0,  0.0, 'g', 'S+')
    testdrive1(visual, Node(8.0, 7.0, 0.0),  1.0, -1.0, 'r', 'R+')
    testdrive1(visual, Node(3.0, 1.0, 0.0), -1.0,  1.0, 'c', 'L-')
    testdrive1(visual, Node(3.0, 4.0, 0.0), -1.0,  0.0, 'y', 'S-')
    testdrive1(visual, Node(3.0, 7.0, 0.0), -1.0, -1.0, 'm', 'R-')
    visual.show("Showing the driving test cases")

def testcosts(visual):
    # Make a move.
    def move(node, forward, steer, comment):
        node = node.nextNode(forward, steer)
        print(node, comment)
        return node
```

```
459
460     # Movements.
461     node = Node(4.0, 4.0, 0.0)
462     print(node, "   starting node")
463     node = move(node,  1,  0, "   after S+")
464     node = move(node,  1,  0, "   after S+")
465     node = move(node,  1,  1, "   after L+")
466     node = move(node,  1,  1, "   after L+")
467     node = move(node,  1, -1, "   after R+")
468     node = move(node,  1, -1, "   after R+")
469     node = move(node, -1,  0, "   after S-")
470     node = move(node, -1,  0, "   after S-")
471
472
473 ##########################################################################
474 #
475 #   Main Code
476 #
477 def main():
478     # Report the parameters.
479     print('Running with step size %.2fm and delta-theta of 360/%d = %.2fdeg' %
480           (dstep, int(2*pi/thetastep), np.degrees(thetastep)))
481     print('So the possible steering angles are 0 or +/- %.2fdeg' %
482           (np.degrees(steerangle)))
483     print('Cost %d per move plus %d per steering and %d per direction change' %
484           (cstep, csteer, creverse))
485
486     # Create the figure.
487     visual = Visualization()
488
489     # Testing!  FIXME: Change to False to run the regular code.
490     if False:
491         testdriving(visual)
492         return
493     if False:
494         testcosts(visual)
495         return
496
497     # Create the start/goal nodes.
498     startnode = Node(xstart, ystart, thetastart)
499     goalnode  = Node(xgoal,  ygoal,  thetagoal)
500
501     # Show the start/goal nodes.
502     visual.drawNode(startnode, color='c', linewidth=2)
503     visual.drawNode(goalnode,  color='m', linewidth=2)
504     visual.show("Showing basic world", 0)
```

```
505
506
507         # Run the planner.
508         print("Running planner...")
509         path = planner(startnode, goalnode)
510
511         # If unable to connect, show what we have.
512         if not path:
513             visual.show("UNABLE TO FIND A PATH")
514             return
515
516         # Print/Show the path.
517         print("PATH found after sampling %d nodes" % nodecounter)
518         print("PATH length %d nodes, %d steps" % (len(path), len(path)-1))
519         for node in path:
520             print(node)
521         visual.drawPath(path, color='r', linewidth=2)
522         visual.show("Showing the path")
523
524     if __name__== "__main__":
525         main()
```